Table des matières

Cours 07 — Traiter un gros volume 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Ce qu'une machine fait déjà · RAW 2
Étape 2 — Le même travail, mille fois plus gros · RAW 3
Étape 3 — Le travail réparti sur les cœurs · RAW 4
Étape 4 — Un fichier par fil · RAW 5
Étape 5 — L'action qui ramène tout au programme · RAW 5
Étape 6 — Quand une machine ne suffit plus · RAW 6
À retenir 7

Cours 07 — Traiter un gros volume

Cours 7/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
		✓ ici

Ce que vous allez faire

Quand le volume grandit, la première question n'est pas de savoir quel moteur distribué installer, mais ce que fait déjà la machine qu'on a. Boréal, au même cours, mesure son historien de 2,4 millions de lignes sur une machine, puis en Spark, le moteur distribué que tout le monde nomme. La Brûlerie n'a que 3 718 lectures de sondes : rien à distribuer. Ce cours prend donc la mesure qui tranche, dans le notebook : un agrégat des courbes et son plan, le même sur un volume mille fois plus gros, le travail réparti sur les cœurs, un fichier écrit par fil. Puis il mesure, dans Python, ce qui coûte vraiment : ramener tout le volume dans un programme.

les courbes de la `RAW` → un agrégat et son plan → mille fois plus de lectures → un fil, puis tous les cœurs → un fichier par fil dans le lac → le volume ramené dans Python, ou le seul résultat

© 2026 iAlexMG. Tous droits réservés.

Aucun fichier du dépôt ne s'écrit dans ce cours : on mesure. Chaque requête se donne en pseudocode, avec des trous ____ à remplir ; le code complet est dans `Codes_07_DE.md`, à ouvrir après avoir essayé.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour Python et Git, qui servent à l'étape 5 ; la seconde pour le notebook, qui s'ouvre à l'étape 1. Le notebook ne fait que lire l'entrepôt : il le garde attaché du début à la fin. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
dir /b plateforme\lac
REM Attendu : bronze.
```

Ce cours part de l'entrepôt chargé jusqu'au 30 juin, tel que le cours 06 le laisse. S'il manque, reprenez la fin du cours 06.

Étape 1 — Ce qu'une machine fait déjà · RAW

Avant de distribuer quoi que ce soit, on mesure ce qu'une seule machine fait. `EXPLAIN ANALYZE`, devant une requête, l'exécute pour de bon, puis rend son plan : les opérateurs que le moteur a enchaînés, les lignes que chacun a rendues et le temps qu'il y a passé, avec le temps total en tête. L'outil s'apprend au Cours 10 de la formation SQL avancé, `Theorie_10_SQL.md`, section 3, sur PostgreSQL ; le plan de DuckDB se dessine autrement, mais se lit pareil, de bas en haut.

1.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « Cours 07 ». Dans sa première cellule, attacher l'entrepôt en lecture seule :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/brulerie.duckdb' AS entrepot (READ_ONLY);
-- Attendu : entrepot paraît sous Attached databases.
```

1.2 Lire le nombre de fils de DuckDB, dans une cellule neuve. `current_setting` rend la valeur d'un réglage de la session.

```
SELECT current_setting('threads') AS fils;
-- Attendu : le nombre de cœurs logiques de votre poste, 12 sur celui où ce cours a été mesuré.
```

Un fil (*thread*) est une suite d'instructions qu'un cœur exécute. DuckDB en ouvre d'office un par cœur logique : sans qu'on lui demande rien, il est prêt à travailler en parallèle.

1.3 Mesurer un agrégat des courbes : par lot, les lectures et la plus haute température du grain. La `RAW` garde tout en texte : la température se convertit avant de se comparer.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- 1. exécuter l'agrégat, et rendre son plan avec ce que chaque opérateur a rendu
____
SELECT lot_id, COUNT(*) AS lectures, MAX(CAST(temp_grain AS DECIMAL(5, 1))) AS temp_max
FROM entrepot.raw_courbes
GROUP BY lot_id;
-- Attendu : Total Time de quelques millisecondes ; de bas en haut, TABLE_SCAN à 3,718 rows,
-- puis HASH_GROUP_BY à 47 rows.
```

Lire le plan de bas en haut :

L'opérateur	Ce qu'il fait	Les lignes qu'il rend
TABLE_SCAN	lit `raw_courbes`, et seulement les deux colonnes nommées sous `Projections`	3 718
PROJECTION	convertit `temp_grain` en `DECIMAL`	3 718
HASH_GROUP_BY	range chaque lecture dans le groupe de son lot, compte et garde le maximum	47

Deux projections `__internal_…` encadrent le regroupement : DuckDB y range le nom du lot dans un nombre, le temps de grouper, puis l'en ressort. Elles ne viennent pas de la requête. Rien dans ce plan ne déplace de lignes d'une machine à l'autre : chez Spark, un `Exchange` y redistribuerait les résultats partiels. Un seul processus, une seule mémoire, et 3 718 lectures agrégées en quelques millisecondes.

Étape 2 — Le même travail, mille fois plus gros · RAW

3 718 lectures, c'est un mois d'un seul torréfacteur. Pour voir ce qu'une machine fait d'un vrai volume, on le fabrique : mille torréfacteurs comme celui de La Brûlerie, pendant le même mois. `range(1000)` rend une table d'une colonne, `range`, qui porte les nombres de 0 à 999 ; un `CROSS JOIN` colle chacune de ses lignes à chaque lecture, sans condition, et donne donc chaque lecture mille fois, une par torréfacteur. `range` s'apprend au Cours 05 de la formation SQL avancé, `Theorie_05_SQL.md`, section 6, où le `CROSS JOIN` sert aussi, section 3. `CREATE OR REPLACE TABLE … AS` crée une table à partir d'une requête, et la remplace si elle existe : Cours 03, `Theorie_03_SQL.md`, section 2.

La table naît dans la mémoire du notebook, pas dans l'entrepôt, qui reste en lecture seule : elle disparaîtra avec le notebook. Les lectures s'y typent une fois pour toutes.

2.1 Créer le gros volume, dans une cellule neuve.

```sql
-- 1. chaque lecture mille fois, une par torréfacteur, typée une fois pour toutes
CREATE OR REPLACE TABLE gros AS
SELECT t.range AS torrefacteur, c.lot_id, CAST(c.seconde AS INTEGER) AS seconde,
       CAST(c.temp_grain AS DECIMAL(5, 1)) AS temp_grain
FROM entrepot.raw_courbes AS c
```


© 2026 iAlexMG. Tous droits réservés.

```
CROSS JOIN ____(1000) AS t;
-- Attendu : 3718000 sous Count.
```

2.2 Mesurer le même agrégat sur les 3 718 000 lectures, dans une cellule neuve.

```
-- 1. le même agrégat par lot, sur le gros volume, déjà typé
EXPLAIN ANALYZE
SELECT lot_id, COUNT(*) AS lectures, MAX(temp_grain) AS temp_max
FROM ____
GROUP BY lot_id;
-- Attendu : Total Time de quelques centièmes de seconde ; TABLE_SCAN à 3,718,000 rows,
-- puis HASH_GROUP_BY à 47 rows.
```

Mille fois plus de lectures, et le temps total passe de quelques millisecondes à quelques centièmes de seconde : un poste ordinaire agrège des millions de lignes sans qu'on voie l'attente. Boréal mesure la même chose sur son historien, 2 405 366 points agrégés en 0,19 s.

Étape 3 — Le travail réparti sur les cœurs · RAW

DuckDB range une table par morceaux d'environ 122 880 lignes. Chaque fil prend un morceau, le traite, puis en prend un autre : c'est la partition de Spark, sans le réseau. Chaque fil regroupe sa part de son côté, puis les résultats partiels se réunissent. Le plan analysé le laisse voir : le temps affiché sous un opérateur additionne celui de tous les fils, et peut donc dépasser le temps total.

3.1 Relire le plan de l'étape 2.2. Attendu : sous `HASH_GROUP_BY`, un temps plus grand que le Total Time, 0.15s contre 0.0265s sur le poste du cours. Plusieurs fils ont regroupé en même temps.

3.2 Passer à un seul fil, dans une cellule neuve. `SET` change un réglage pour toute la session du notebook.

```
-- 1. un seul fil pour toutes les cellules qui suivent
SET threads = ____;
-- Attendu : 0 rows returned.
```

Puis relancer la cellule de l'étape 2.2 : cliquer dedans, puis Ctrl+Entrée. Attendu : un Total Time plusieurs fois plus long, 0.122s contre 0.0265s sur le poste du cours, et sous `HASH_GROUP_BY` un temps plus petit que le total : un seul fil a tout fait.

Douze fils n'ont pas rendu l'agrégat douze fois plus vite, mais quatre à cinq fois : découper le travail, puis réunir ce que chaque fil a regroupé, coûte aussi. Ajouter des cœurs, ou des machines, ne divise jamais tout à fait la durée par leur nombre.

3.3 Rendre les fils, dans une cellule neuve.

```
RESET threads;
-- Attendu : 0 rows returned.
```

Puis relancer la cellule de l'étape 1.2. Attendu : le même nombre qu'à la sous-étape 1.2.

© 2026 iAlexMG. Tous droits réservés.

Étape 4 — Un fichier par fil · RAW

Un moteur qui lit sur plusieurs fils peut aussi écrire sur plusieurs fils : chaque fil écrit sa part dans son propre fichier, sans attendre les autres. Spark le fait à chaque écriture, et c'est pourquoi il écrit un dossier de fichiers plutôt qu'un fichier ; DuckDB le fait sur demande, par l'option `PER_THREAD_OUTPUT` du `COPY` vu au cours 02. Le dossier `essai` va dans le lac, hors du dépôt, le temps du cours.

4.1 Écrire le gros volume en Parquet, un fichier par fil, dans une cellule neuve.

```
-- 1. le gros volume dans plateforme/lac/essai, chaque fil dans son fichier
COPY gros TO 'plateforme/lac/essai' (FORMAT parquet, ____);
-- Attendu : 3718000 sous Count.
```

4.2 Compter les lectures de chaque fichier.

```
-- 1. chaque fichier écrit, et ses lectures
SELECT PARSE_FILENAME(filename) AS fichier, COUNT(*) AS lectures
FROM read_parquet('plateforme/lac/essai/*.parquet', filename = true)
GROUP BY fichier
ORDER BY fichier;
-- Attendu : un fichier par fil, de data_0.parquet à data_11.parquet sur le poste du cours (data_10
-- vient avant data_2 : l'ordre est alphabétique), chacun de quelques centaines de milliers de lectures.
```

Les parts ne sont pas égales, et elles changent d'un passage à l'autre : chaque fil a écrit les morceaux qu'il a pris, dans l'ordre où il les a pris. Le dossier, lui, porte toujours les 3 718 000 lectures.

4.3 Peser le dossier.

```
SELECT COUNT(*) AS fichiers, SUM(size) AS octets FROM read_blob('plateforme/lac/essai/*.parquet');
-- Attendu : autant de fichiers qu'en 4.2, et moins d'un million d'octets.
```

Moins d'un mégaoctet pour 3 718 000 lectures : Parquet compresse bien une colonne qui répète mille fois la même courbe. Et un dossier de fichiers se lit comme un seul, par un joker : l'étape suivante le lit ainsi.

Étape 5 — L'action qui ramène tout au programme · RAW

Dans un programme Python, DuckDB est paresseux, comme Spark : `duckdb.read_parquet(…)` décrit une lecture sans rien lire, et `.aggregate(…)` ajoute un agrégat à la description. Le travail ne part que quand on demande un résultat, par une action :

L'action	Ce qu'elle rend
`.show()`	quelques lignes à l'écran ; DuckDB s'arrête dès qu'il les a

© 2026 iAlexMG. Tous droits réservés.

L'action	Ce qu'elle rend
`.fetchone()`	une seule ligne, en tuple Python
`.fetchall()`	toutes les lignes, chacune faite d'objets Python

Le piège est la dernière : l'action qui ramène tout le volume dans le programme. Le moteur finit sa part en un rien de temps ; Python, lui, fabrique un objet par valeur. `time.perf_counter()` rend un instant, en secondes : la différence de deux instants donne une durée.

5.1 Montrer quelques lignes, dans la première fenêtre.

```
python -c "import duckdb; duckdb.read_parquet('plateforme/lac/essai/*.parquet').show()"
REM Attendu : un tableau de 20 lignes, les 10 premières et les 10 dernières, puis
REM ? rows (>9999 rows, 20 shown) : DuckDB s'est arrêté sans tout compter.
```

5.2 Demander l'agrégat au moteur, chronomètre en main.

```
python -c "import time, duckdb; t = time.perf_counter();
print(duckdb.read_parquet('plateforme/lac/essai/*.parquet').aggregate('COUNT(*), MAX(temp_grain)').fetchone(),
round(time.perf_counter() - t, 2), 's')"
REM Attendu : (3718000, Decimal('219.1')) 0.01 s, ou quelques centièmes de seconde.
```

5.3 Tout ramener dans Python. Comptez une dizaine de secondes, davantage sur un poste plus lent.

```
python -c "import time, duckdb; t = time.perf_counter(); lignes =
duckdb.read_parquet('plateforme/lac/essai/*.parquet').fetchall(); print(len(lignes), 'lignes ramenées en',
round(time.perf_counter() - t, 1), 's')"
REM Attendu : 3718000 lignes ramenées en une dizaine de secondes.
```

La même lecture a pris près de mille fois le temps de l'agrégat, et de la mémoire avec : mesuré sur le poste du cours, le programme est monté à 1,3 Go, pour un dossier de moins d'un mégaoctet. Sur un vrai gros volume, c'est l'erreur de mémoire, qui ne nomme pas son coupable. La parade est la même chez Spark, où l'action s'appelle `collect()` : une action rend un résultat, pas un jeu de données. On compte, on agrège, on écrit, et on ne ramène que ce qu'on va lire.

Étape 6 — Quand une machine ne suffit plus · RAW

Un moteur distribué se justifie par ce qui ne tient plus sur une machine, jamais par principe. Les signes se mesurent, et La Brûlerie n'en montre aucun :

Le signe	Ce qu'il veut dire	Chez La Brûlerie
le jeu ne tient plus en mémoire, même par morceaux	le disque devient le goulot	mille torréfacteurs tiennent en moins d'un mégaoctet de Parquet

© 2026 iAlexMG. Tous droits réservés.

Le signe	Ce qu'il veut dire	Chez La Brûlerie
le temps imparti est dépassé alors que tous les cœurs travaillent	il n'y a plus rien à gagner sur une machine	3 718 000 lectures agrégées en quelques centièmes de seconde
la donnée vit déjà répartie sur des machines distantes, et personne ne veut la rapatrier	le calcul va vers la donnée, pas l'inverse	un dossier sur un poste
le traitement doit reprendre après la panne d'une machine, sans tout refaire	un moteur distribué rejoue les tâches perdues	le pipeline se relance en une commande, et le journal ne double rien (cours 05 et 06)

Spark tourne sur Java, et demande sous Windows un utilitaire de plus, `winutils` : il reste à Boréal, qui le lance dans un conteneur et le compare au moteur d'une machine sur le même travail.

6.1 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt. La table `gros` part avec lui.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer. Le notebook « Cours 07 » reste enregistré.

6.2 Effacer le dossier d'essai, dans la première fenêtre. `rmdir /s /q` efface un dossier et tout ce qu'il contient, sans demander de confirmation.

```
rmdir /s /q plateforme\lac\essai
dir /b plateforme\lac
REM Attendu : bronze, comme au début du cours.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean ; aucun fichier du dépôt n'a changé.
```

À retenir

- On mesure ce qu'une machine fait avant de distribuer : `EXPLAIN ANALYZE` dit ce que chaque opérateur a rendu, et le temps qu'il y a passé.
- DuckDB répartit seul le travail sur tous les cœurs, et écrit un fichier par fil quand on le lui demande : 3 718 000 lectures s'agrègent en quelques centièmes de seconde.
- Ce qui coûte, c'est de ramener le volume dans un programme : une action rend un résultat, pas un jeu de données.

© 2026 iAlexMG. Tous droits réservés.